

EXPLORE: Exploration with Guided Search for Analog Topology Generation using Language Models

Guanglei Zhou¹, Chen-Chia Chang¹, Yikang Shen², Jonathan Ku¹, Isaac Jacobson¹, Jingyu Pan¹,
Yiran Chen¹, Xin Zhang³

¹Electrical and Computer Engineering, Duke University, Durham, USA

²MIT-IBM Watson AI Lab, Cambridge, USA ³IBM T. J. Watson Research Center, Yorktown Heights, USA

¹{guanglei.zhou, chenchia.chang, jonathan.ku, isaac.jacobson, jingyu.pan, yiran.chen}@duke.edu

²yikang.shen@ibm.com ³xzhang@us.ibm.com

Abstract

Automating analog circuit topology design is essential to reduce the extensive manual effort required to meet increasingly diverse and customized application demands. Recent advances have applied sequence-to-sequence fine-tuning on pretrained language models to directly generate circuit topologies from user specifications in a single pass. However, these one-shot generation methods failed to generate complex circuits due to their exponentially growing search spaces and limited training datasets. In this paper, we present **EXPLORE**, a search-enhanced framework that integrates simulator-guided Monte Carlo Tree Search (MCTS) with transformer-based decoding to enable test-time scaling for analog topology generation. By leveraging language-model priors and bypassing high-confidence structural tokens, EXPLORE allocates expensive simulator budget primarily toward topology-altering decisions during search. On a 6-component benchmark at a tight tolerance of 0.01, EXPLORE raises the success rate from 12% for one-shot generation and 33% for a sampling-and-filter baseline to 65%, and lowers MSE by over 20× relative to sampling-and-filter under the same search budget. These results establish EXPLORE as the first framework to integrate structured test-time search with LM decoding for analog topology generation, and a practical step toward scaling LLM-driven design automation.

1 Introduction

Analog circuit topology design sits at the heart of modern electronic systems, enabling everything from efficient power conversion to high-speed signal processing. As device requirements proliferate, varying voltage-conversion ratios, efficiency targets, and performance specifications, the burden on designers to craft bespoke topologies grows heavier. Traditional workflows remain largely manual, demanding extensive domain expertise and hundreds of simulation iterations per new requirement, which in turn prolongs development cycles and delays time-to-market. To meet these challenges, automating the topology design process has become essential: by embedding search and learning methods directly into the design flow, engineers can rapidly explore vast design spaces, reduce iteration counts, and accelerate the creation of optimized analog circuits.

Early works [6, 16, 26] leverage reinforcement learning (RL) or Bayesian optimization to discover valid topologies using simulation feedback. These methods reduce evaluation costs and produce functional designs, but suffer from two key limitations: (1) they must restart search or retrain policies for every new specification, and (2)

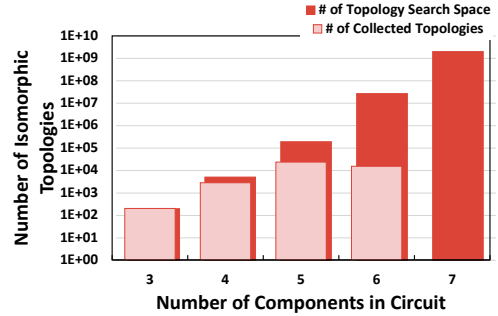


Figure 1: Topology space grows combinatorially with circuit complexity, while collected topologies remain limited, resulting in increasingly sparse design-space coverage [3, 7].

they have only demonstrated success on small, relaxed 3–5 component circuits. Without access to prior knowledge across tasks, such methods remain inefficient and unsuitable for practical usage.

LaMAGIC [3] reframed topology generation as a sequence-to-sequence problem for autoregressive language models, introducing several text-based circuit formulations. Trained on a corpus of 132k 3–5 component converter topologies, LaMAGIC achieved strong results within this regime. However, as Figure 1 illustrates, the topology search space grows exponentially with component count. At six components, a good dataset coverage becomes impractical: with an average simulation time of 9 seconds per topology, enumerating the full design space would require over 2000 CPU-days. Consequently, LaMAGIC [3] struggles to transfer knowledge from 345 components to six, and scaling further to complex circuits with 8–10 components becomes infeasible. This gap highlights a fundamental bottleneck: existing approaches, constrained by limited datasets, cannot scale topology generation on their own. Overcoming this limitation requires not only new computing paradigms, such as search-enhanced test-time scaling techniques, but also novel dataset collection strategies that efficiently harvest high-quality training data in larger, more complex circuit spaces.

In this work, we propose **EXPLORE**, a search-enhanced language model framework for automated analog topology generation. EXPLORE integrates simulator-guided Monte Carlo Tree Search (MCTS) with transformer-based decoding to leverage inference-time compute for generating higher-quality and more scalable analog circuit topologies. Building on text-based circuit formulations, our

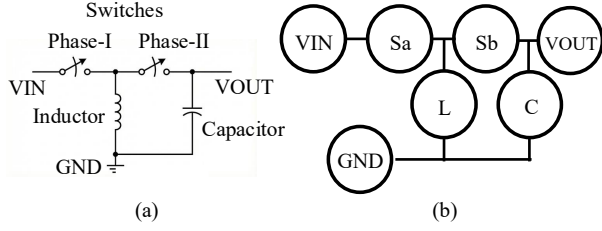


Figure 2: (a) An example power converter circuit and (b) its corresponding graph representation.

framework combines pretrained language-model priors with simulator feedback to guide exploration toward valid and high-performance circuit structures. In addition, we construct a large-scale dataset of higher-complexity analog topologies to study the scalability of language-model-based topology generation beyond prior works.

The main contributions of this work are as follows:

- **First test-time-scaling framework.** EXPLORE is the first work to bring test-time scaling to analog topology generation, along two axes: an LM-guided MCTS as a test-time decoder, and the same search reused as a model-native data-collection engine.
- **Structured-token filtering.** We bypass structural tokens in the text-based circuit formulation via p-filtering ($p = 0.99$), skipping 24–48% of expansive simulation trials and making scaling to higher-complexity circuits computationally feasible.
- **Superior generation success and query efficiency.** At a tight tolerance of 0.01 on 6-comp, EXPLORE raises the success rate from 12% for one-shot LaMAGIC and 33% for its sampling-and-filter variant to 65%, with over $20\times$ lower MSE than sampling-and-filter under the same search budget and faster convergence than our MCTS-Base ablation within 100 generations.
- **Scalability beyond one-shot generation method.** EXPLORE enables generation at 7, 8, and 9 components, a regime where one-shot LM decoding collapses to near-zero success and which prior 3–6 component datasets do not cover.

2 Preliminaries

2.1 Analog topology design

In this work, we address the same problem as LaMAGIC [3]: generating customized power converters that meet specific voltage conversion ratios and efficiency targets. The **voltage conversion ratio** is the output-to-input voltage ratio, while **power conversion efficiency** is the output-to-input power ratio. The **duty cycle** (a number between 0-1) is a design parameter, which controls the ON time of switches, affecting performance. We use five discrete duty cycles: 0.1, 0.3, 0.5, 0.7, 0.9. We represent circuits as hypergraphs G with vertices V and hyperedges E . Vertices include three terminals (input V_{IN} , output V_{OUT} , and ground GND) and four component types (capacitors C , inductors L , and switches S_a, S_b). Hyperedges define connections between components and terminals. Figure 2 shows an example converter with its hypergraph representation.

Problem Statement: Given vertices V , target conversion ratio r , and efficiency η , our model generates connections E and selects duty cycle s to create a circuit meeting both performance requirements.

2.2 Search- and optimization-based methods

Classical analog topology generation treats the problem as a black-box search over a combinatorial graph space guided by simulator feedback. [6] models power converter design as a sequential decision process using a UCT-based RL tree with physics-guided pruning, achieving up to 67% fewer SPICE calls than genetic or random search. [26] applies deep RL to op-amp synthesis, combining symbolic analysis and memorization to converge to feasible designs within hours but requiring retraining for each circuit class. [16] pairs a variational autoencoder with Bayesian optimization to identify spec-compliant topologies more efficiently than graph-grammar engines, and [7] accelerates the same search loop with a graph-transformer SPICE surrogate. More recently, ADO-LLM [23] replaces a hand-tuned acquisition function with in-context learning over an LLM to seed Bayesian optimization for analog sizing. Across this line of work, each new specification still triggers a fresh search or policy retraining, and the demonstrated regime is small.

2.3 Language model-based methods

A recent paradigm shift reframes topology generation as a conditional sequence generation task: given a target specification, the model decodes a circuit description token-by-token. AnalogCoder [13], AnalogCoder-Pro [14], AnalogXpert [24], and Artisan [4] prompt or fine-tune general or domain LLMs to emit SPICE-style code or operate over curated subcircuit libraries, primarily targeting general circuit *families*. Closest to our setting, LaMAGIC [3] fine-tunes an encoder-decoder transformer to map a target (v^*, η^*) to a converter topology in one autoregressive pass. Subsequent works have optimized this paradigm by compressing adjacency representations to $O(|V|)$ [1, 2], and applying RL fine-tuning [21]. CktGNN [5] pairs adjacency with a graph VAE, and AnalogGenie [8]/AnalogGenie-Lite [9] adopt Eulerian-circuit and device-pin representations. All of these remain *one-shot* generators: they expend their entire model capacity on a single decoded sequence, with no mechanism for spending extra compute when the first attempt is structurally invalid or off-spec. For complex circuit creation that requires iterative refinement and simulator feedback, this one-shot paradigm is highly inefficient. Diverging from all prior work, EXPLORE introduces test-time scaling through a structured search framework guided by simulator feedback. EXPLORE keeps the text-based formulation but reframes generation as a guided search, drawing on the search-augmented LLM decoding literature [10, 15, 22] to spend additional inference-time compute with simulator feedback on tokens where the model is uncertain.

2.4 Complex power converter datasets: prior corpora and challenges

The lack of sufficiently large analog circuit datasets continues to hinder the development of AI-based generative methods for automating analog IC design. Prior corpora such as AnalogGenie [8], Align [12], CktGNN [5], AMSNet [20], AMSnet-KG [19], and AICircuit [17] prioritize breadth, covering diverse circuit families with only thousands of examples per circuit type. Despite the substantial manual effort required to curate these samples from textbooks and datasheets, the per-type sample sizes remain too small for a model to learn the internal dynamics of complex circuits. In contrast, we prioritize on

power converters and explore scaling towards higher-component topologies within a single circuit class.

Scaling a power converter dataset to higher component counts, however, introduces three compounding challenges: (1) the learning task becomes more complex, so substantially more training samples are required; (2) the fraction of invalid or useless topologies (e.g., disconnected graphs or near-zero-efficiency circuits) rises sharply; and (3) the probability that a purely random arrangement yields an efficient converter shrinks rapidly as the search space grows combinatorially. These obstacles motivate the two-stage construction pipeline detailed in Section 3.2.

3 Search-enhanced language model framework

We build on LaMAGIC [3], whose text-based formulations for circuit generation include the float-input adjacency-matrix formulation (FM), which achieved the best MSE on 6-component circuits. As shown in Figure 3(a), FM represents circuit connections as an adjacency matrix over a hypergraph, with distinct tokens <no edge>, <edge 1>, <edge 2>, and <both edges> indicating the connection type between each vertex pair. A property we exploit below is that more than half of the tokens in an FM sequence are *structural*: they keep the formulation syntactically legal but carry no topological choice (highlighted in Figure 3(a)), which is the opening that p-filtering exploits.

Although our pipeline follows the PUCT-style LM-guided MCTS template used for code generation [15, 25], three properties of analog topology generation reshape almost every design choice. (i) *The search target is a graph, not text.* A circuit topology is an undirected (hyper)graph; the FM token stream is just one serialization of its adjacency matrix, so the actual decision space lives over edges between component pairs rather than over the surface tokens that the LM emits. (ii) *Not all tokens are decisions.* As noted above, structural tokens of FM carry no topological choice; in code generation, by contrast, nearly every token changes program semantics. Spending simulator budget on forced tokens is pure waste, which makes p-filtering structurally well-motivated rather than a generic engineering speedup. (iii) *Each evaluation is orders of magnitude more expensive.* A single NGSPICE simulation costs roughly 4 seconds to a few minutes, versus milliseconds for a typical unit test in code-generation settings. Simulator budget, not search depth, is the dominant constraint: LM priors focus expansion on plausible edges, p-filtering removes forced moves, and we beam-complete partial topologies before simulating so every simulator call has a chance of returning useful reward.

As illustrated in Figure 3(b), EXPLORE addresses these constraints with a single MCTS loop guided by transformer LM priors. Each iteration selects a node with PUCT, expands either by auto-committing a high-confidence (forced) token via p-filtering or by spawning top- k children at a genuine decision point, evaluates by beam-completing the partial topology and simulating it in NGSPICE, and backpropagates the resulting reward. The following subsections detail each step.

3.1 LM-guided topology search

Algorithm 1 summarizes the procedure; we detail each phase below.

Selection. We apply PUCT-style action priors (in the spirit of AlphaCode [15] and PG-TD [25]) to LM-guided MCTS over circuit tokens: the language-model probability of the child action token enters the UCB exploration term as the action prior $P(a \mid s)$, biasing selection toward likely and under-explored continuations. A tunable exploration constant c controls the trade-off, with higher c encouraging broader search.

Expansion. After selecting a node, we first apply **p-filtering**: if the language model assigns probability $\geq p$ to its top-1 token (we use $p=0.99$), we auto-commit that token and append it to the current node without spawning new children. This pattern is empirically dominated by structural tokens of the FM representation that carry no topological choice. Note that p-filtering is the *opposite* of top- p (nucleus) sampling: nucleus sampling *prunes low-probability* tokens to limit diversity, whereas p-filtering *skips high-probability* tokens that the model is already certain about, so as not to spend simulator budget on forced moves. When no token exceeds the threshold, we apply **top- k sampling** to generate multiple child nodes, each representing an extended partial topology.

Evaluation. Since partial topologies cannot be directly simulated, we use beam search to complete the sequence from the current node, using a predefined prefix and beam width b . The completed topology is then evaluated with NGSPICE to obtain circuit-level performance metrics (output voltage v and conversion efficiency e), which are compared to the target (v^*, e^*) using the reward

$$r = 0.5 \cdot \max(0, 1 - |v - v^*|) + 0.5 \cdot \max(0, 1 - |e - e^*|), \quad (1)$$

clipped to $[0, 1]$. The 0.5/0.5 weighting is an unweighted average of voltage and efficiency error; tuning this weighting is left to future work.

Backpropagation uses the standard PUCT-style mean backup: each ancestor accumulates the leaf reward and exposes *node.value* = $\frac{1}{N} \sum_i r_i$, the running average over the N rollouts that pass through it.

Algorithm 1 MCTS-based token generation

Input: root: initial state; c : UCB exploration parameter;

1: k : max children per node; b : beam search width;

2: p : threshold for structural token filtering

Output: Best sequence from MCTS

3: Initialize tree with root node

4: **for** $i = 1$ to max_rollouts **do**

5: $\text{node} \leftarrow \text{root}$

▷ Selection

6: **while** node has children **do**

7: $\text{node} \leftarrow \text{Select child using UCB}$

8: **while** $\text{len}(\text{node.top-}p \text{ tokens})=1$ **do**

▷ p-filtering

9: $\text{node} \leftarrow \text{CONCAT}(\text{node}, \text{next_tokens})$

10: $\text{next_tokens} \leftarrow \text{top-}k \text{ tokens}$

▷ Expansion

11: **for** $\text{token} \in \text{next_tokens}$ **do**

12: Add new child node for token to tree

13: $\text{sequence} \leftarrow \text{Perform beam search}$

▷ Evaluation

14: $r \leftarrow \text{Obtain reward via simulation}$

15: Backpropagate(node, r)

▷ Backpropagation

16: **return** sequence with the highest reward

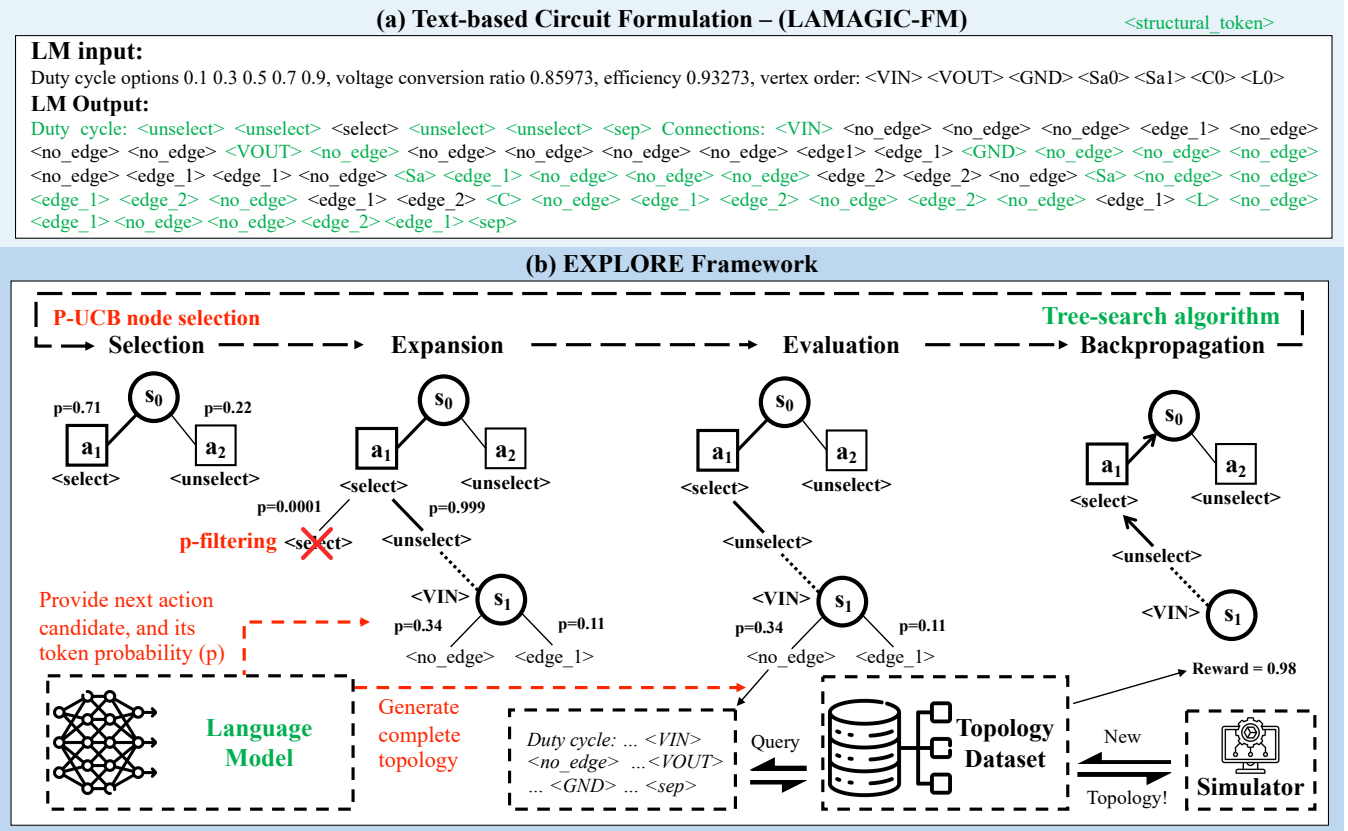


Figure 3: (a) A circuit example of float-input adjacency-based matrix formulation for the edge generation task, highlighting its inefficiency due to structural tokens. (b) Illustration of the EXPLORE framework pipeline through a step-by-step example of leveraging an MCTS algorithm to guide the Transformer generation for analog circuit topology.

3.2 Two-stage dataset construction

To overcome the three scaling challenges identified in Section 2.4, we construct our power-converter corpus in two stages: we first expand the random-enumeration pipeline to bootstrap higher-component data, then close the loop by using the trained model itself, guided by the search procedure above, as a high-quality data generator.

Stage 1: expanding the random-enumeration pipeline. We extend the random topology generator of LaMAGIC [3] beyond the 3–5 component regime to 6–10 components. We predefine a pool of candidate components (capacitors and inductors at various values, and the two switch types), sample the target number of components, add the three terminals V_{IN} , V_{OUT} , and GND, and draw a random connected topology; each candidate is then simulated in NGSPICE and only valid circuits are retained. Randomizing both the component selection and the connectivity yields topological diversity, but the yield of this pipeline degrades with scale exactly as challenges (2) and (3) predict: for the 7-component set we generated 300,000 circuits and only 255,707 (85.2%) were structurally valid, and the vast majority of those exhibit low efficiency. This makes random enumeration alone impractical for building large high-quality datasets at higher component counts.

Stage 2: model-native data collection. Once a model has acquired a basic understanding of the formulation from Stage-1 data,

we reuse the trained model together with the EXPLORE framework as a data generator. Because the LM prior biases expansion toward plausible edges and simulator feedback steers the search toward high-efficiency targets, model-native collection attains a far higher useful-sample yield than random enumeration: on 6-component circuits it cuts the fraction of sub-2%-efficiency topologies from 66.1% to 18.2% and raises the fraction above 90% efficiency from 8.3% to 23.3% (analyzed in detail in Appendix B.3). Using this pipeline we assemble, to our knowledge, the largest 6-component converter dataset to date (350k samples), substantially exceeding the 120k 3–5 component public release of LaMAGIC [3].

4 Experimental results

4.1 Experiment setup

Baseline algorithms. We compare EXPLORE with four decoding baselines. **Greedy** is the one-shot generation used in LaMAGIC. **Beam Search** uses Transformer beam search (beam size 20) without simulator feedback. **Sampling and filtering (S+F)** samples a set of topologies (temperature 1.2, top- k with $k=3$ to avoid invalid tokens), simulates each to measure v_{out} and efficiency, and returns the candidate closest to the target; this mirrors AlphaCode [15] and a baseline in PG-TD [25]. **MCTS-Base** is our MCTS variant that

Method	1 k training data			32 k training data		
	Success Rate ($t = 0.01$)	MSE (Voltage)	MSE (Efficiency)	Success Rate ($t = 0.01$)	MSE (Voltage)	MSE (Efficiency)
Greedy	0.05	0.22	0.28	0.12	0.262	0.174
Beam Search	0.13	0.043	0.029	0.32	0.027	0.023
Sampling + Filter	0.20	0.0070	0.0081	0.33	0.007	0.008
MCTS-Base	0.17	0.0154	0.0093	0.31	0.008	0.010
Ours ($c=4$)	0.52	0.00150	0.00135	0.65	0.00029	0.00019

Table 1: Performance on the 6-comp at threshold $t = 0.01$ for both 1 k and 32 k training-data budgets. All methods use up to 100 Transformer generations.

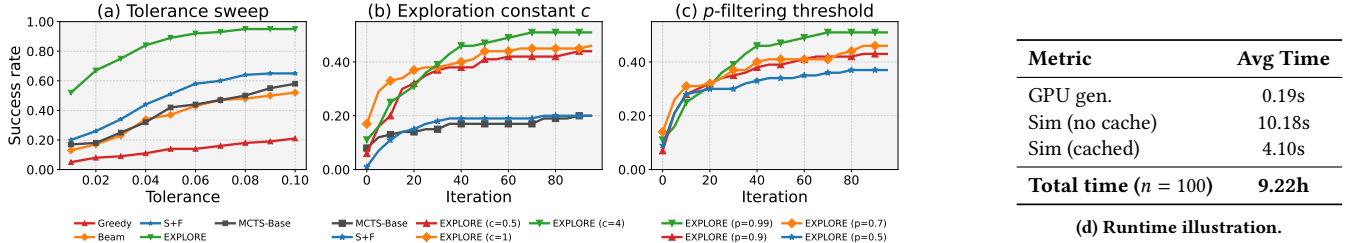


Figure 4: Comparison of success rates and runtime analysis on the 6-comp benchmark (FM-1k). (a): final success rate under varying error tolerances. (b): success rate over iterations at $t=0.01$ under different exploration constants $c \in \{0.5, 1, 4\}$. (c): success rate over iterations at $t=0.01$ under different p -filtering thresholds $p \in \{0.5, 0.7, 0.9, 0.99\}$ at fixed $c = 4$. (d) Runtime breakdown and total evaluation time for EXPLORE.

uses UCB for node selection but ignores LLM token probabilities, treating all equal-count children uniformly, thereby isolating the benefit of LLM-guided priors.

Models and datasets. Large amount of data for circuits with a higher number of components can be difficult to obtain. To reuse existing knowledge, similar to the LaMAGIC’s setting, we extend models trained with 3,4,5-components to be finetuned with 1k and 32k 6-component circuits and leverage our decoding algorithm to evaluate. We follow LaMAGIC [3] to use an encoder-decoder transformer structure with Flan-T5-base pretrained weights. We add a shared linear layer to replace the word embedding layer for numeric inputs. We train the model via conditional generation to learn the mapping between input-output pairs. Model trained with n samples using FM formulation is denoted as FM- n . As the full 7k LaMAGIC validation set is costly and dominated by low-performing circuits, our primary benchmark is **6-comp**: 100 high-efficiency samples spanning various conversion ratios. To confirm the gains are not a selection artifact, we also evaluate **6-comp-random-100**, 100 samples drawn uniformly from the original set (Appendix B.4).

Evaluation metrics. Our primary metric is success rate: the percentage of generated circuits meeting the preset target (v^*, e^*). For each target and search budget n , the method generates up to n candidates and retains the best. A circuit succeeds if its simulated conversion ratio and efficiency (v, e), obtained via NGSPICE [18], both fall within tolerance t of the target:

$$|v - v^*| \leq t \quad \text{and} \quad |e - e^*| \leq t.$$

We report two variants: (1) **tolerance-based success rate**, with $t \in \{0.01, 0.02, \dots, 0.10\}$, and (2) **iteration-wise success rate**, which tracks success under strict tolerance ($t = 0.01$) as the number of

generated candidates increases. Candidates that fail to compile or simulate count as failures. We additionally report mean squared errors (MSEs) for the voltage conversion ratio and efficiency to quantify deviation among valid circuits.

4.2 Generation results on 6-component circuit

Comparison with baselines. For a fair comparison, we evaluate the best topology found by the different decoding algorithms when they use the same number of Transformer generations. All methods are evaluated under the same success criterion on 6-comp.

Results are shown in Table 1. Our method consistently outperforms all the other baselines on the 6-comp validation set for various tolerance thresholds. Overall, these results confirm that our algorithm indeed generates better topologies for the target voltage and efficiency. As shown in Table 1, EXPLORE largely outperforms the one-shot generation methods, reaching a success rate of 0.65 (Ours) versus 0.33 (S+F) when using the FM-32k model at a tight tolerance of 0.01 on 6-comp. S+F uses the same number of transformer generations but is overall outperformed by EXPLORE. Comparing with MCTS-Base, this confirms that the LLM probability guidance is crucial in the node selection stage. A runtime breakdown for our framework is also provided in Figure 4d.

To illustrate the exploration efficiency of various decoding methods, Figure 4b plots the strict-tolerance success rate ($t=0.01$) as a function of iteration count. For EXPLORE, we fix the top-k sampling budget at $k=3$ with a single beam ($b=1$) and sweep the P-UCB exploration constant c . We observe that $c=1$ yields faster gains in the very early iterations, while a larger c ($c=4$) drives more exploration and ultimately achieves the highest success. In contrast, without the LLM token probability guidance, MCTS Baselines explores less

Method	Success Rate ($t = 0.01$)				MSE (Voltage)				MSE (Efficiency)			
	7-comp	8-comp	9-comp	10-comp	7-comp	8-comp	9-comp	10-comp	7-comp	8-comp	9-comp	10-comp
Greedy	0.02	0.02	0.04	0.00	0.763	0.904	0.906	1.019	0.338	0.348	0.377	0.591
Sampling + Filter	0.03	0.09	0.09	0.00	0.269	0.372	0.188	0.928	0.231	0.190	0.094	0.476
MCTS-Base	0.09	0.09	0.11	0.00	0.115	0.154	0.481	0.917	0.045	0.044	0.124	0.475
Ours ($c=4$)	0.21	0.26	0.16	0.00	0.125	0.009	0.046	0.838	0.024	0.008	0.027	0.432

Table 2: Performance at threshold $t = 0.01$ with varying number of components(7 $\rightarrow$ 10). All methods use up to 100 Transformer generations.

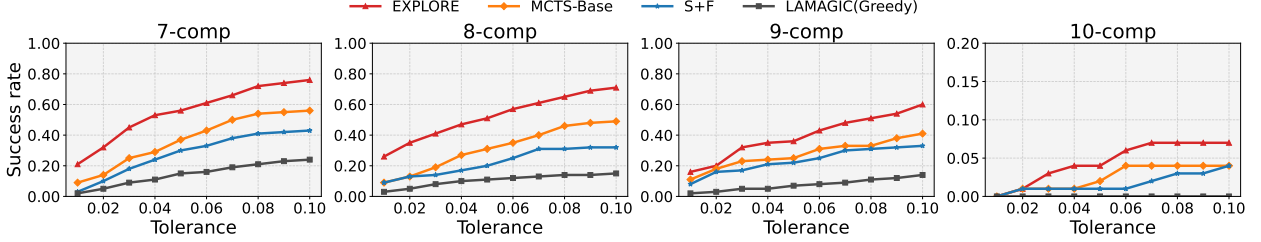


Figure 5: Comparison of generation results on validation sets with varying number of components (7 $\rightarrow$ 10), for Greedy, Sampling + Filtering (S+F), MCTS-Base, and our EXPLORE methods.

effectively and converges to sub-optimal results with other baseline sampling and filtering.

We further sweep the p-filtering threshold p at fixed $c=4$ (Figure 4c). Since p-filtering auto-commits tokens whose model probability already exceeds p , a smaller p skips more tokens and saves more budget per iteration. The sweep reveals a clear trade-off: aggressive filtering ($p=0.7$) produces the fastest gains in the first few iterations, but skipping tokens the model is less confident about discards genuine decision content and plateaus lower, with the most aggressive setting ($p=0.5$) capping at a success rate of 0.37. Setting $p=0.99$ auto-commits only near-certain structural tokens, preserving search quality and reaching the highest final success (0.51) while still skipping 24–48% of token expansions.

4.3 Generation results on 7–10 component circuit

A central claim of this work is that test-time search scales more gracefully than one-shot decoding as circuit complexity grows. To test this, we retrain the baseline with 4K samples (1K each from 6–9 components) and build 7/8/9/10-comp validation sets; details are in Appendix B.1. As shown in Figure 5, all methods degrade as components increase from 7 to 10, but ours consistently leads across the 7/8/9/10 benchmarks. Structured search (EXPLORE and MCTS-Base) clearly outperforms the S+F approaches, underscoring its importance for graph-generation tasks such as circuit topology design. Quantitatively (Table 2), EXPLORE achieves 10 $\times$ /13 $\times$ /4 $\times$ success-rate gains over the greedy baseline on 7/8/9-comp, with up to 100 $\times$ lower voltage MSE.

Unseen 10-component circuits. The rightmost panel of Figure 5 reports a 10-component set entirely outside the 6–9 training range. As expected, few valid circuits are generated, yet two points stand out: (1) even without pre-training, EXPLORE still produces

higher-complexity circuits, indicating a scalability trend across component counts absent from training; and (2) its relative advantage persists, remaining the top method in this untrained regime.

Overall, these results support our claim that test-time search scales more effectively than baseline strategies as complexity grows, making it well suited to higher-component and future large-scale designs. We stress that they reflect a *scalability trend* rather than practical readiness, as absolute success rates remain modest beyond 8 components. Further results extending EXPLORE to other analog types (op-amps, bandgap references) via AnalogGenie, and a discussion of MCTS as a data-collection method, are deferred to Appendix B.2 and B.3.

5 Conclusion

In this work, we propose EXPLORE, a search-enhanced language model framework for analog circuit topology generation. EXPLORE tightly integrates simulator-guided Monte Carlo Tree Search (MCTS) with transformer-based decoding, capitalizing on both the learned priors of pretrained models and the dynamic feedback of circuit simulators to navigate vast design spaces. We adapt MCTS to the text-based adjacency-matrix formulation via PUCT-style action priors from the language model, top- k expansion, and p-filtering that auto-commits high-confidence tokens so simulator budget concentrates on genuine decision points.

Extensive experiments across varying data regimes and difficulty settings confirm that EXPLORE achieves higher success rates and lower error metrics than prior RL-based search methods and language model decoding strategies such as sampling and filtering. We further demonstrate its utility as a high-quality data collection engine for scaling analog datasets in low-coverage regimes. Future work includes discovering more efficient circuits, generalizing to more complex analog designs, and developing search-friendly formulations that reduce structural token overhead.

Acknowledgments

This work was supported in part by NSF grant No. 2112562.

References

- [1] Chen-Chia Chang, Wan-Hsuan Lin, Yikang Shen, Yiran Chen, and Xin Zhang. 2025. LaMAGIC2: Advanced Circuit Formulations for Language Model-Based Analog Topology Generation. In *Proceedings of the 42nd International Conference on Machine Learning* (Vancouver, Canada) (ICML '25). JMLR.org. <https://openreview.net/forum?id=Y0zXGw0GUk>
- [2] Chen-Chia Chang, Wan-Hsuan Lin, Yikang Shen, Guanglei Zhou, Yiran Chen, and Xin Zhang. 2026. LaMAGIC: Advanced Circuit Formulations for Language-Model-based Topology Generation for Analog Integrated Circuits. *ACM Transactions on Design Automation of Electronic Systems* 31, 5 (April 2026), 1–21. doi:10.1145/3799428
- [3] Chen-Chia Chang, Yikang Shen, Shaoze Fan, Jing Li, Shun Zhang, Ningyuan Cao, Yiran Chen, and Xin Zhang. 2024. LaMAGIC: Language-Model-based Topology Generation for Analog Integrated Circuits. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (ICML '24). JMLR.org, Article 241, 10 pages. <https://proceedings.mlr.press/v235/chang24c.html>
- [4] Zihao Chen, Jiangli Huang, Yiting Liu, Fan Yang, Li Shang, Dian Zhou, and Xuan Zeng. 2024. Artisan: Automated Operational Amplifier Design via Domain-specific Large Language Model. In *Proceedings of the 61st ACM/IEEE Design Automation Conference* (San Francisco, CA, USA) (DAC '24). Association for Computing Machinery, New York, NY, USA, Article 39, 6 pages. doi:10.1145/3649329.3655903
- [5] Zehao Dong, Weidong Cao, Muhan Zhang, Dacheng Tao, Yixin Chen, and Xuan Zhang. 2023. CktGNN: Circuit Graph Neural Network for Electronic Design Automation. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=NE2911Kq1sp>
- [6] Shaoze Fan, Ningyuan Cao, Shun Zhang, Jing Li, Xiaoxiao Guo, and Xin Zhang. 2021. From Specification to Topology: Automatic Power Converter Design via Reinforcement Learning. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 1–9. doi:10.1109/ICCAD51958.2021.9643552
- [7] Shaoze Fan, Haoshu Lu, Shun Zhang, Ningyuan Cao, Xin Zhang, and Jing Li. 2024. Graph-Transformer-based Surrogate Model for Accelerated Converter Circuit Topology Design. In *Proceedings of the 61st ACM/IEEE Design Automation Conference* (San Francisco, CA, USA) (DAC '24). Association for Computing Machinery, New York, NY, USA, Article 172, 6 pages. doi:10.1145/3649329.3656258
- [8] Jian Gao, Weidong Cao, Junyi Yang, and Xuan Zhang. 2025. AnalogGenie: A Generative Engine for Automatic Discovery of Analog Circuit Topologies. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=jCPak79Kev>
- [9] Jian Gao, Weidong Cao, and Xuan Zhang. 2025. AnalogGenie-Lite: Enhancing Scalability and Precision in Circuit Topology Discovery through Lightweight Graph Modeling. In *International Conference on Machine Learning (ICML)*. <https://openreview.net/forum?id=KRk0WTII0I>
- [10] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with Language Model is Planning with World Model. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 8154–8173. doi:10.18653/v1/2023.emnlp-main.507
- [11] Levente Kocsis and Csaba Szepesvári. 2006. Bandit Based Monte-Carlo Planning. In *Machine Learning: ECML 2006*, Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 282–293. doi:10.1007/11871842_29
- [12] Kishor Kunal, Meghna Madhusudan, Arvind K Sharma, Wenbin Xu, Steven M Burns, Ramesh Harjani, Jiang Hu, Desmond A Kirkpatrick, and Sachin S Sapatnekar. 2019. ALIGN: Open-source Analog Layout Automation from the Ground Up. In *Proceedings of the 56th Annual Design Automation Conference 2019*, 1–4. doi:10.1145/3316781.3323471
- [13] Yao Lai, Sungyoung Lee, Guojin Chen, Souradip Poddar, Mengkang Hu, David Z. Pan, and Ping Luo. 2025. AnalogCoder: Analog Circuit Design via Training-Free Code Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, 379–387. doi:10.1609/aaai.v39i1.32016
- [14] Yao Lai, Souradip Poddar, Sungyoung Lee, Guojin Chen, Mengkang Hu, Bei Yu, Ping Luo, and David Z. Pan. 2026. AnalogCoder-Pro: Unifying Analog Circuit Generation and Optimization via Multi-modal LLMs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* (2026). doi:10.1109/TCAD.2026.3673493
- [15] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level Code Generation with AlphaCode. *Science* 378, 6624 (Dec. 2022), 1092–1097. doi:10.1126/science.abq1158
- [16] Jialin Lu, Liangbo Lei, Jiangli Huang, Fan Yang, Li Shang, and Xuan Zeng. 2023. Automatic Op-Amp Generation From Specification to Layout. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (2023), 4378–4390. doi:10.1109/TCAD.2023.3296374
- [17] Asal Mehrdadfar, Xuzhe Zhao, Yue Niu, Sara Babakniya, Mahdi Alesheikh, Hamidreza Aghasi, and Salman Avestimehr. 2024. AICircuit: A Multi-Level Dataset and Benchmark for AI-Driven Analog Integrated Circuit Design. In *Machine Learning and the Physical Sciences Workshop, NeurIPS*. <https://arxiv.org/abs/2407.18272>
- [18] Paolo Nenzi and Holger Vogt. 2011. *Ngspice Users Manual Version 23*. <https://pkgs.fedoraproject.org/repo/extras/ngspice/ngspice23-manual.pdf/eb0d68eb463a41a0571757a00a5b9f9d/ngspice23-manual.pdf> Accessed: 2023.
- [19] Yichen Shi, Zhuofu Tao, Yuhao Gao, Tianjia Zhou, Cheng Chang, Yaxing Wang, Bingyu Chen, Genhao Zhang, Alvin Liu, Zhiping Yu, Ting-Jung Lin, and Lei He. 2025. AMSnet-KG: A Netlist Dataset for LLM-based AMS Circuit Auto-Design Using Knowledge Graph RAG. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 30, 6, Article 94 (2025). doi:10.1145/3736166
- [20] Zhuofu Tao, Yichen Shi, Yiru Huo, Rui Ye, Zonghang Li, Li Huang, Chen Wu, Na Bai, Zhiping Yu, Ting-Jung Lin, et al. 2024. AMSNet: Netlist Dataset for AMS Circuits. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–5. doi:10.1109/LAD62341.2024.10691781
- [21] Prashanth Vijayaraghavan, Luyao Shi, Ehsan Degan, Vandana Mukherjee, and Xin Zhang. 2025. AutoCircuit-RL: Reinforcement Learning-Driven LLM for Automated Circuit Topology Generation. In *International Conference on Machine Learning (ICML)*. <https://openreview.net/forum?id=NvYwrQbzOb>
- [22] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://openreview.net/forum?id=5Xc1ecxO1h>
- [23] Yuxuan Yin, Yu Wang, Boxun Xu, and Peng Li. 2024. ADO-LLM: Analog Design Bayesian Optimization with In-Context Learning of Large Language Models. In *International Conference on Computer-Aided Design (ICCAD)*. doi:10.1145/3676536.3676816
- [24] Haoyi Zhang, Shizhao Sun, Yibo Lin, Runsheng Wang, and Jiang Bian. 2025. AnalogXpert: Automating Analog Topology Synthesis by Incorporating Circuit Design Expertise into Large Language Models. In *2025 International Symposium of Electronics Design Automation (ISED)*, 772–777. doi:10.1109/ISED65950.2025.11100627
- [25] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. 2023. Planning with Large Language Models for Code Generation. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=Lr8cOOtYbfl>
- [26] Zhenxin Zhao and Lihong Zhang. 2022. Analog Integrated Circuit Topology Synthesis With Deep Reinforcement Learning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 12 (2022), 5138–5151. doi:10.1109/TCAD.2022.3153437

Appendix

A Further Explanation of methodologies

A.1 P-UCB node selection

To guide exploration during tree traversal, we extend the standard Upper Confidence Bound (UCB) [11] strategy by incorporating the token probability predicted by the language model. The P-UCB score for each child node is computed as:

$$exploration = \sqrt{\frac{\log(node.visit_count)}{child.visit_count}}$$

$$score = exploitation + c \times exploration * child.token_probability$$

where $node.visit_count$ records the number of times the current (parent) node has been visited, $exploitation$ is $child.value$, and language-model probability are assigned at the parent to the action token that produced this child, i.e., the action prior $P(a | s)$ in classical PUCT terminology. The hyperparameter c controls the trade-off between exploration and exploitation; in our experiments we sweep $c \in \{0.5, 1, 4\}$. The selection step picks the child with the highest score; the resulting selection, expansion, evaluation, and backpropagation steps are described in Section 3.

B Additional experimental setup and results

B.1 Model training details and compute resources

The Flan-T5-base model consists of 12 transformer layers in both the encoder and decoder. Each layer includes key and value projections with a dimensionality of 64, a feed-forward network with a hidden size of 2048, and employs 12 attention heads. Overall, the model contains approximately 248 million parameters.

To adapt the tokenizer for our specific application, we add custom tokens to its vocabulary. For the FM task, the following tokens are introduced: <sep>, <duty 0.1>, <duty 0.3>, <duty 0.5>, <duty 0.7>, <duty 0.9>, VIN, VOUT, GND, Sa, Sb, C, L, <no edge>, <edge 1>, <edge 2>, and <both edges>.

Training is conducted on a machine equipped with eight NVIDIA A5000 GPUs. The language model is trained over 30 epochs using the AdamW optimizer with an initial learning rate of 3×10^{-4} . A cosine learning rate schedule is applied with 300 warmup steps. The batch size is set to 128, L2 regularization is applied with a strength of 10^{-5} , and the dropout rate is set to 0.1.

B.2 Evaluation on Other Analog Component Types

Experimental Setup. We follow the open-source release of AnalogGenie, using its publicly available model checkpoint on HuggingFace to faithfully reproduce its analog circuit generation behavior. A generated circuit is considered invalid if any device has incomplete pin connectivity (e.g., an NMOS missing one or more of S/D/G/B), or if an electrical short is detected, such as a direct connection between VDD and VSS. We apply our framework using AnalogGenie’s original text-based formulation and conduct experiments under the same generation setting. The reward function is defined as a combination of circuit legality and device count, encouraging valid topologies while discouraging unnecessarily complex designs.

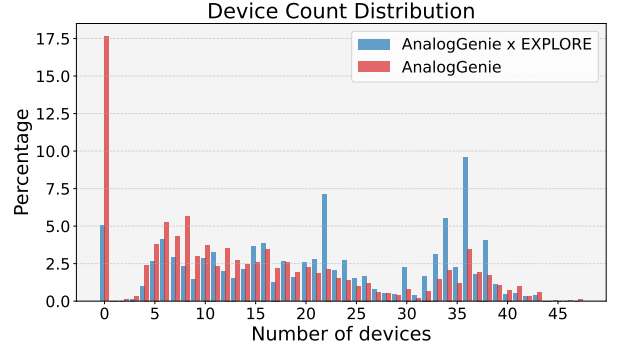


Figure 6: Device count distribution for the first 10000 generated circuits produced by AnalogGenie and AnalogGenie x EXPLORE. Samples with zero devices correspond to invalid generations, including circuits with shorted connections or incomplete pin definitions.

Generation Results. To evaluate the generality of the proposed formulation, we conduct an ablation study on AnalogGenie for generating additional analog components, including op-amps and bandgap references. Both AnalogGenie and AnalogGenie x EXPLORE are evaluated under the same generation budget (10000 samples) for a fair comparison. As shown in Fig. 6, AnalogGenie produces a large fraction of zero-device samples, corresponding to invalid circuits caused by pin shorts or incomplete connections. In contrast, AnalogGenie x EXPLORE substantially reduces the invalid generation rate and shifts the distribution toward higher device counts. This indicates improved topology exploration and more stable generation behavior, demonstrating that EXPLORE generalizes effectively to more complex analog circuit families.

B.3 MCTS as an effective data collection method

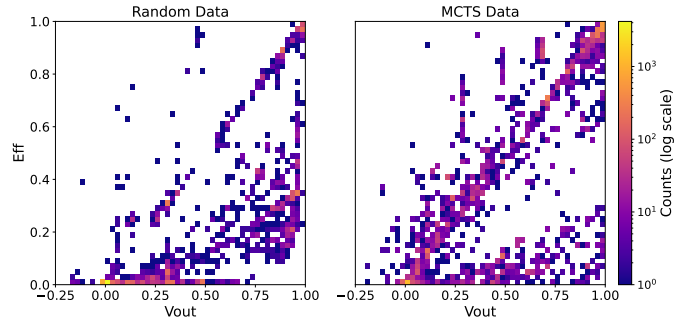


Figure 7: The Vout vs efficiency distribution of our model collected dataset vs. random connection generated dataset.

MCTS combined with generative models represents a powerful yet often overlooked approach for high-quality data collection in complex design spaces. Traditional random generation methods become exponentially ineffective as design complexity grows, with the vast topology search space severely diminishing the probability of discovering valid, high-performance configurations. Our empirical analysis quantifies this limitation: in a random generation of

10,000 6-component circuits, 66.13% exhibited efficiency below 2%, rendering them practically unusable for both application and training purposes. In contrast, our MCTS-based approach significantly mitigates this inefficiency problem, reducing the proportion of low-performing circuits to just 18.2%. More importantly, our method substantially enhances the discovery of high-quality designs, generating 23.27% of circuits with efficiency exceeding 90%—nearly three times higher than the 8.3% achieved through random generation. The complete efficiency distribution illustrated in Figure 7 demonstrates this substantial quality difference. This shows that beyond immediate circuit applications, our approach can facilitate an efficient mechanism for collecting a high-quality dataset for automatic discovery of unconventional topology and enable further training for the language models.

B.4 Unbiased Random Evaluation on 6-component

The 6-comp subset is mined for high-efficiency targets, which could in principle favor search-based methods. To rule out this selection bias, we evaluate on samples drawn uniformly at random from the full 7k LaMAGIC validation set.

Method	# Train Data	Success Rate ($t = 0.01$)	MSE (Voltage)	MSE (Efficiency)
Greedy	1 000	0.21	0.33	0.16
Beam Search	1 000	0.41	0.033	0.022
Sampling + Filter	1 000	0.70	0.0216	0.0053
MCTS-Base	1 000	0.62	0.0044	0.0016
Ours (c=4)	1 000	0.83	0.00016	0.00057
Greedy	32 000	0.37	0.195	0.165
Beam Search	32 000	0.51	0.025	0.013
Sampling + Filter	32 000	0.70	0.022	0.005
MCTS-Base	32 000	0.67	0.029	0.005
Ours (c=4)	32 000	0.84	0.00006	0.00003

Table 3: Performance on the 6-comp-random-100 split (100 samples drawn uniformly at random from the 7k LaMAGIC validation set) at threshold $t = 0.01$ for both 1k and 32k training-data budgets. All methods use up to 100 Transformer generations. EXPLORE retains the top success rate and lowest MSE, showing the main-table gain is not an artifact of the 6-comp selection.

We first report **6-comp-random-100**, a 100-sample uniform draw evaluated under the same metrics as the main 6-comp table (Table 3). EXPLORE remains the top method at both training budgets, reaching 0.84 success at $t=0.01$ with the FM-32k model versus 0.70 for S+F, with the lowest voltage and efficiency MSE.

To confirm the trend at larger scale, we further evaluate **6-comp-random-500** (Table 4). EXPLORE is the top method at $t=0.01$ (0.73 success vs. 0.59 for MCTS-Base, 0.48 for S+F, and 0.24 for Greedy) and also attains the lowest voltage and efficiency MSE by more than an order of magnitude, confirming that the gain reported in Sec. 4.1 is not an artifact of the hard-subset construction. To keep the table comparable to the 6-comp setting without separately rerunning argmax decoding on the random subset, the Greedy row is populated from the first per-target sample of the S+F generations.

Method	Success Rate ($t = 0.01$)	MSE (Voltage)	MSE (Efficiency)
Greedy	0.24	0.27	0.23
Sampling + Filter	0.48	0.19	0.15
MCTS-Base	0.59	0.0026	0.0026
Ours (c=4)	0.73	0.00063	0.00038

Table 4: Performance on the 6-comp-random-500 split (500 samples drawn uniformly at random from the 7k LaMAGIC validation set) at threshold $t = 0.01$. Greedy is taken as the first per-target sample from the S+F generations. EXPLORE retains the top success rate and lowest MSE, confirming the main-table gain is not an artifact of the 6-comp selection.

B.5 Effect of LLM Probability Guidance

Within our tree structure framework, we incorporate probability guidance during node selection to leverage the LLM’s capabilities in directing tree-based sampling. To validate the effectiveness of this LLM-provided probability guidance, we conducted a comparative analysis by modifying the MCTS baseline from standard UCB node selection to our version of P-UCB node selection. Figure 8 confirms that the probability guidance from P-UCB is useful. However, our method still outperforms MCTS(P-UCB) through the implementation of node shrinking, which effectively addresses the challenges posed by structural tokens in circuit formulation. This advantage is particularly pronounced during the early exploration iterations.

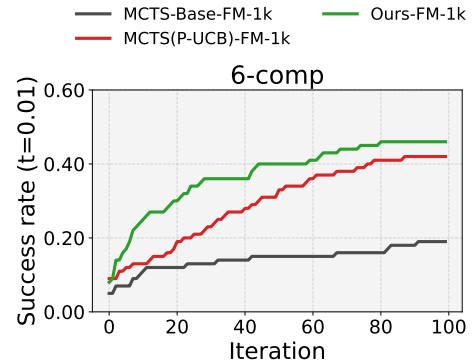


Figure 8: Success rate of our method with MCTS (Baseline) and its MCTS(P-UCB) variants.